

Data Structures and Algorithms

Geetha Lekshmy V
Aswathy R Nair
Department of Computer Science
Amrita Vishwa Vidyapeetham

Linear Search – Running Time

Algorithm 1: Search a value *key* from a list of n numbers

Input: List, *lst* of n numbers, a value to be searched, *key*
Output: True if *key* is in list, False otherwise

```
1 i ← 1
2 while i ≤ n do
3     if lst[i] = key then
4         return True
5     else
6     i ← i + 1
7 return False
```

$T(n) = 5n+4$

$T(n) \varepsilon \ \theta(n)$

- rate of growth/order of growth of T(n) is studied.
- only the leading term of the formula is considered.
- all constants and lower order terms are ignored, it becomes insignificant as input size increases.
- Worst case running time of linear search can be written as $\Theta(n)$.

Growth of Functions

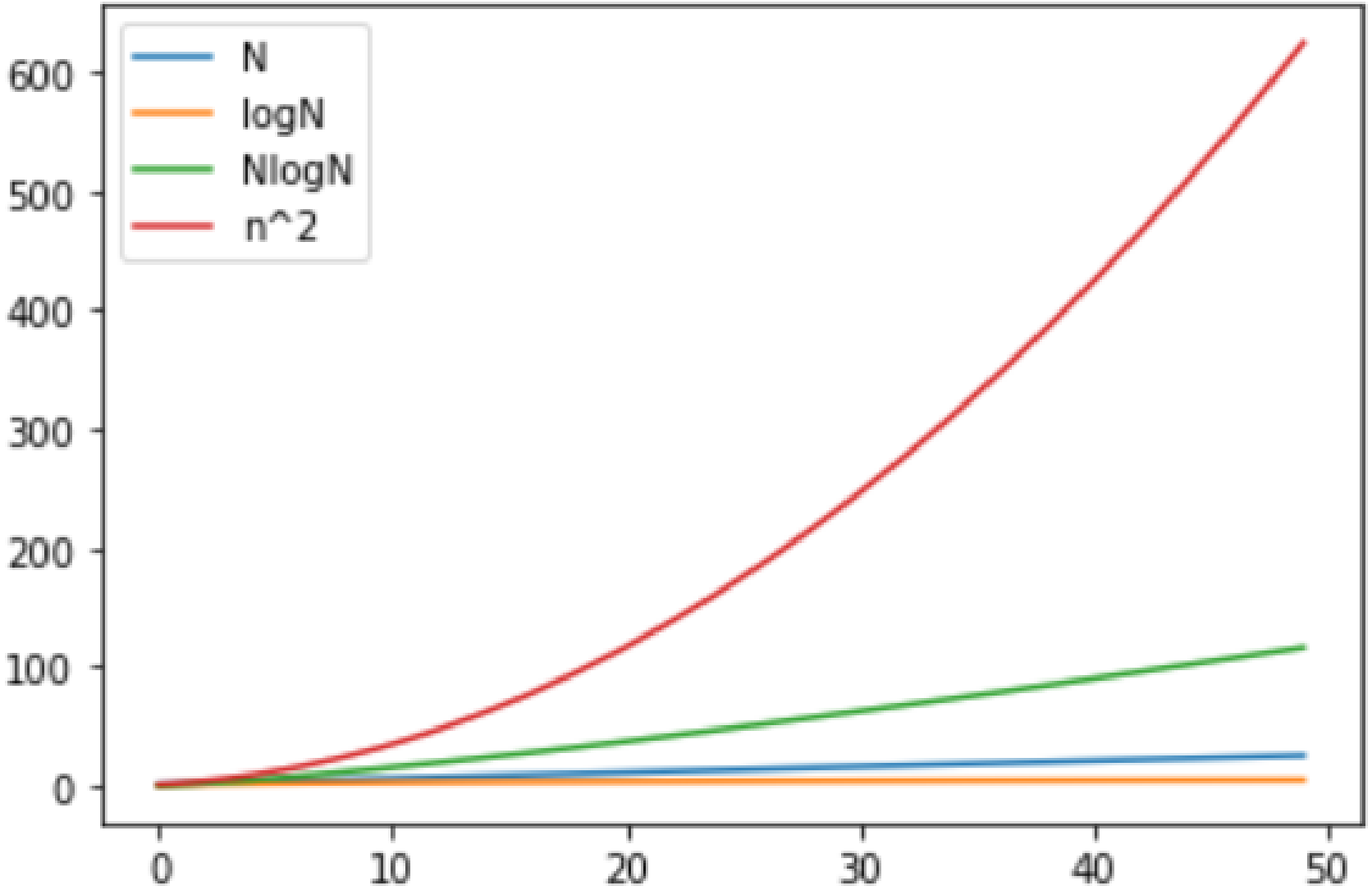
n	$\log_2 n$	n	n $\log_2 n$	n^2	n^3	2^n	n!
10	3.32	10	33.3	100	1000	1024	3628800
100	6.64	100	664	10000	1000000	$1.3 * 10^{30}$	$9.33 * 10^{157}$
1000	9.9657	1000	9965.7	1000^2	1000^3	$1.07 * 10^{301}$	1000!

Table 1: Values of different functions

Log n grows slowly where as n! grows rapidly.

Common Algorithmic Running Times :-
Constant, Logarithmic, Linear, Polynomial, Exponential, Factorial.

Growth of functions



Orders of Growth

- Order of growth of running time of an algorithm is important, when input size is large.
- Asymptotic efficiency – how running time increases as the size of input increases without bound.
- O(big Oh), ϴ(big theta), Ω(big omega) – Notations for expressing orders of growth.

Asymptotic Notations –Theta

- $\theta(g(n))$ – set of functions with same order of growth as g(n)
- $n^3 \in \theta(n^3)$
- $2n^2+5n+4 \in \theta(n^2)$
- $n^2 \notin \theta(Nlogn)$

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Asymptotic Notations –Big O

- O(g(n)) – is a set of functions with same or smaller order of growth as g(n) as n goes to infinity.
- $T(n) = 5n+4$
  $T(n) \in O(n)$
- $n^2 \in O(n^3)$
- $Nlogn \in O(n^2)$
- $n^3 \notin O(n^2)$

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Asymptotic Notations -omega

- $\Omega(g(n))$ – set of functions with larger or same order of growth as g(n)
- $n^3 \in \Omega(n^2)$
- $5n+4 \notin \Omega(n^2)$
- $N^2 \in \Omega(Nlogn)$

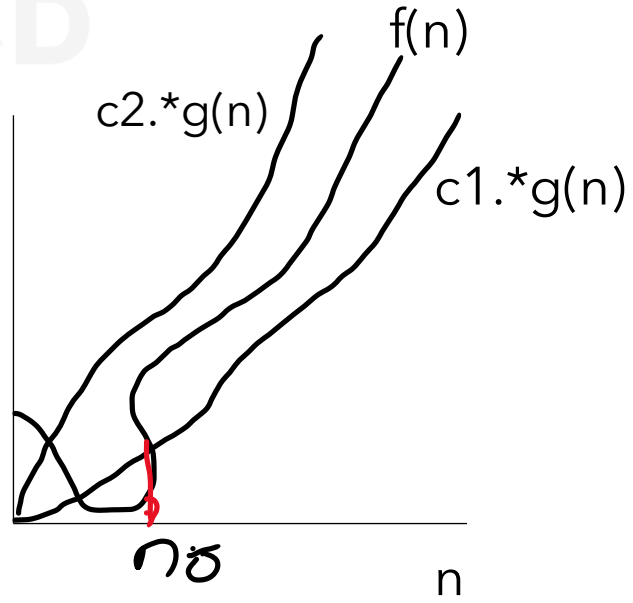
AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Big – Theta Notation (θ)

Definition

A function f(n) belongs to the set, θ (g(n)) if there exist positive constants c1 and c2 such that it can be sandwiched between c1*g(n)and c2*g(n), for sufficiently large n.

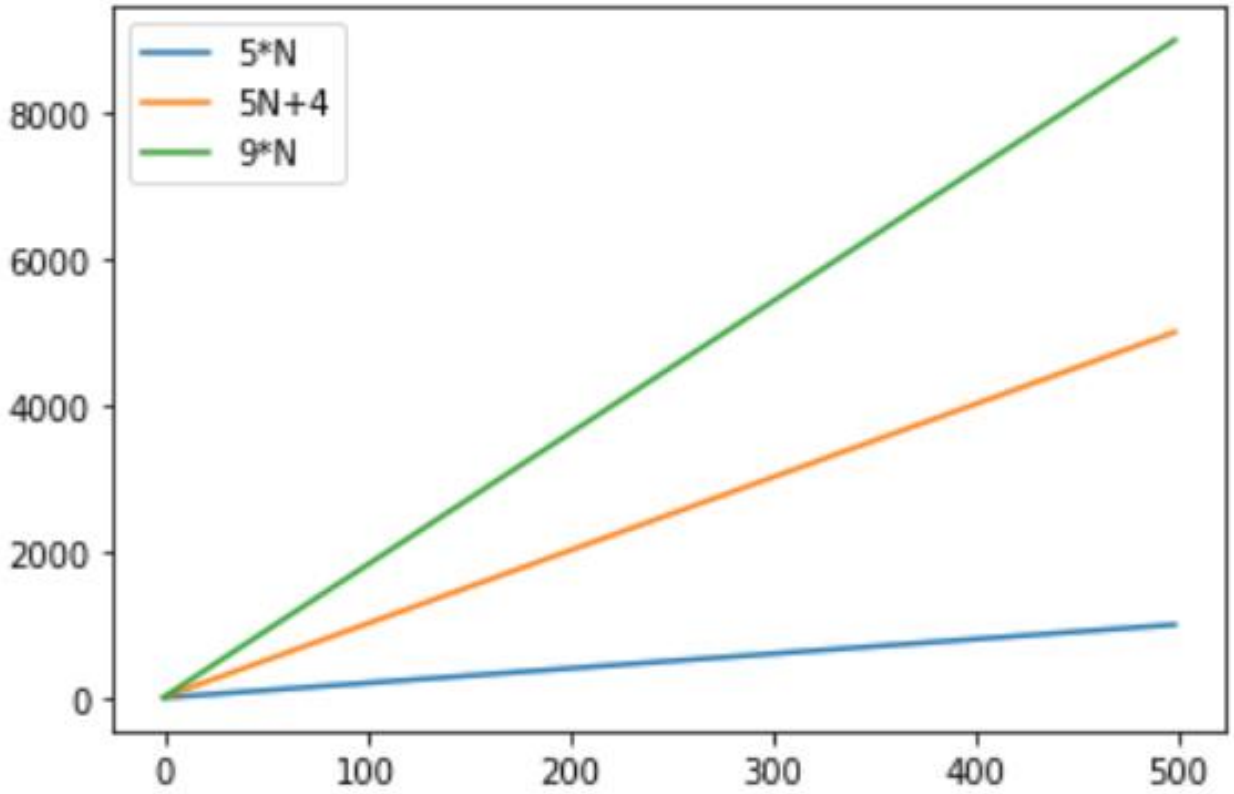
$F(n) \varepsilon \theta(g(n))$ OR $f(n) = \theta(g(n))$



Big – Theta Notation (θ)

Example:- Prove 5n+4 = θ(n)

- Find out c1>=0, c2>=0 and n_0 >= 0 so that the following equality holds.
- 0<= c1 * n <= 5n+4 <= c2*n .
- For n>=1, and c1=5, c2=9 , this equality holds.
- This proves 5n+4 = θ(n).

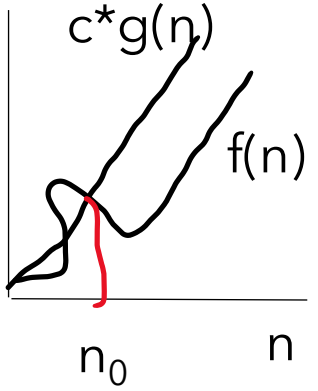


Big – O Notation

Definition

A function f(n) belongs to the set, O(g(n)) if there exist positive constants c and n_0 such that 0<= f(n) <=c*g(n) for all n >= n_0.

F(n) ε O(g(n)) OR f(n) = O(g(n))



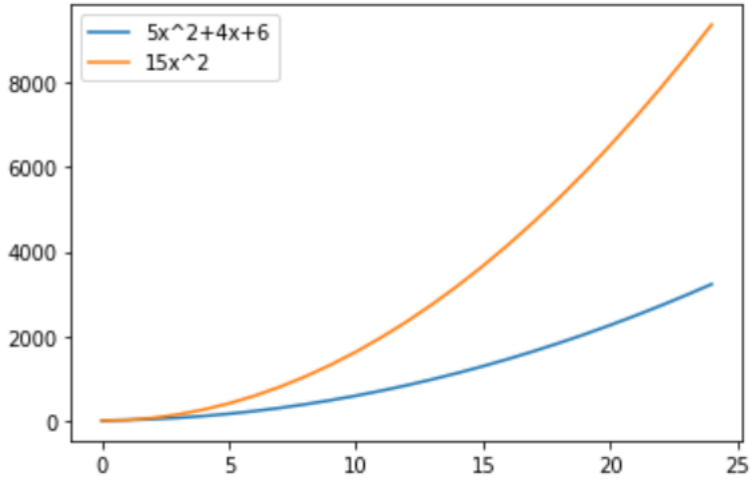
AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

- O-notation gives an asymptotic upper bound on a function.
- f(n)= θ(g(n)) implies f(n) = O(g(n)) since , θ notation is a stronger notion than O-notation.
- O-notation is used to bound worst case running time of an algorithm

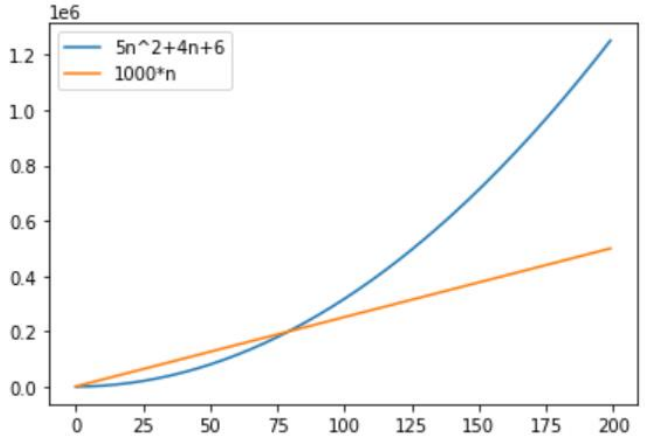
Big – O Notation

Example:- Prove $5n^2+4n+6 = O(n^2)$

- Find out c>=0 and n_0 >= 0 , so that the following equality holds.
- $0 <= 5n^2+4n+6 <= c* n^2$.
- For c=15 then this equality holds, for n>=1.
- This proves $5n^2+4n+6 = O(n^2)$.
- Is $5n^2+4n+6 = O(n)$??



NO



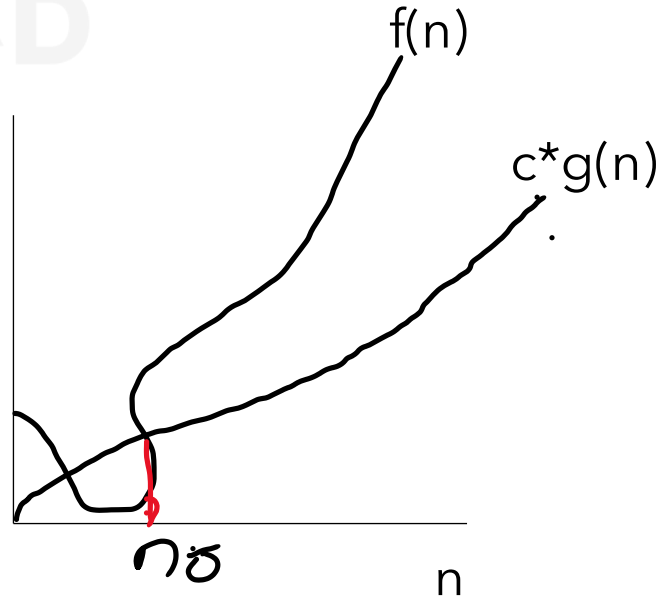
Big – omega Notation (Ω)

Definition

A function f(n) belongs to the set, Ω (g(n)) if there exist positive constants c and n_0 such that c*g(n) <= f(n), for n>= n_0.

f(n) ε Ω(g(n)) OR f(n) = Ω(g(n))

Ω-notation gives lower bound of a function

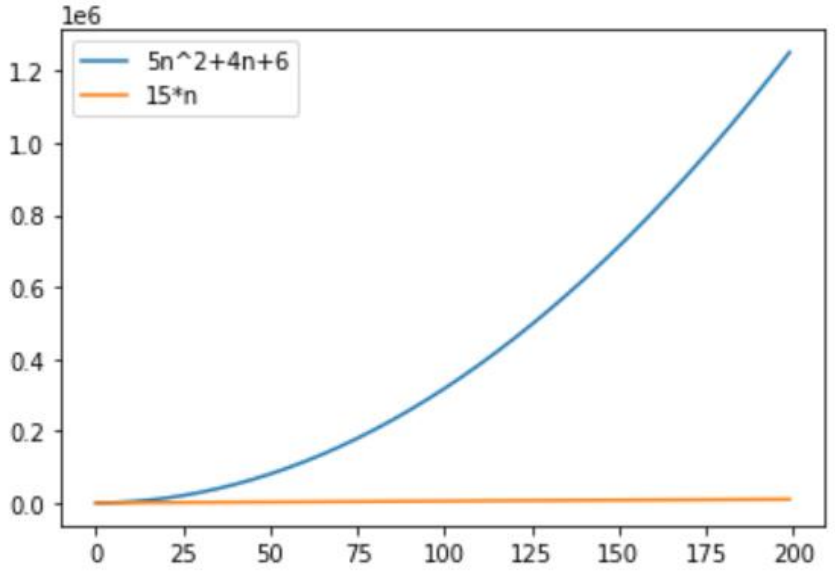


Big – omega Notation (Ω)

Example:- Prove $5n^2+4n+6 = \Omega(n)$

- Find out c>=0 and n_0 >= 0 , so that the following equality holds.
- $0 <= c^* n <= 5n^2+4n+6$.
- For c=15 then this equality holds, for n>=1.
- This proves $5n^2+4n+6 = \Omega(n)$.
- Is $4n+6 = \Omega(n^2)$??

NO



Analysis of Algorithm – Example -1

```
Compare(a, b){
If  a==b then                          →   comparison - 1
    print("a and b are same")          →   print -1
Else if a<b then                       →   comparison -2
    print("b is larger")               →    print-1
Else                                   →   comparison-2
   print("a is larger)                →    print-1
}
```

T(n) = 2+1 =3 comparisons, so $\theta(1)$ →constant time

For conditional statements, take the time complexity which is larger.

Analysis of Algorithm – Example -2

```
Sum(n){
        s=0                 → assignment operation -1
        i=1                 →assignment operation -1
        while(i<=n){        →comparison – n+1
                s=s+i       → assignment and addition – 2(operations) * n
                i=i+1       → 2n
        }
        print(s)            →print -1
}
```

$T(n) = 2+(n+1)+2n+2n+1 = 5n+4 \in \theta(n)$

$T(n) = \theta(n)$

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Analysis of Algorithm – Bubble Sort

Data: A set of n integers in an array A, $n \geq 1$
Result: Set of n integers in the array A, arranged in ascending order.

```
i ← 0 ;
while i ≤ n − 2 do
    swapped ← false;
    j ← 0 ;
    while j ≤ n − 2 − i do
        ;                    /* Swap contents of A[j] and A[j+1] */
        if A[j] > A[j + 1] then
            tmp ← A[j];
            A[j] ← A[j + 1];
            A[j + 1] ← tmp;
            swapped ← true;
        end
        j ← j + 1 ;
    end
    ;    /* no swap indicates, array is sorted and loop can be
     terminated */
    if swapped = false then
        break
    end
    i ← i + 1 ;
end
```

Algorithm 1: Bubble Sort

i=0

j =0,1,2…n-2

i=1

j =0,1,2…n-3

i=2

j = 0, 1…n-4

…….

i=n-2

j =0

No.of iterations of **inner loop** =

n-1+n-2+n-3+n-4+…+1

Sum 0f 1..n-1 = (n)(n-1)/2

= $(n^2-n)/2 \in \theta(n^2)$

$T(n) = \theta(n^2)$

Analysis of Algorithm - Examples

```
for (i = 0; i < n; i = i+2){ // Loop gets executed n/2 times
        print(i)
}
```

Ans:- T(n) = **θ(n)**

```
for (i = 1; i < n; i = i*2){
print(i)        // i =1,2,4,8...2^k  , loop terminates when 2^k = n, K=log2n

}
```

Ans:- T(n)= **$\theta(\log_2(n))$**

Analysis of Algorithm - Examples

```
for (i = 0; i < n; i++){      // Outer loop: n times,
  for(j = 1; j < n; j = j*2){  // Inner loop: logn times
      print(j)
  }
}
```

Ans:- T(n) = **θ(nlogn)**

```
for (i = 1; i <= n; i++){    // Outer loop: n times,
  for(j = 1; j < i*n; j++){  // Inner loop: i*n times
                             // T(n) = 1n + 2n + ... + n*n
  }                          // = n(1 + 2 + .. + n) =n *( n(n+1)/2)
                                                    ∈ θ(n^3)
}
```

Ans:- T(n) = **θ(n^3)**

$(1 + 2 + .. + n) = n(n+1)/2$



Analysis of Algorithm - Examples

```
for (i = 1; i <= n; i++){   // Nested Loop takes Θ(n^2) time
    for(j = 1; j <= n; j++){
        print(i+j)
    }
  }

for(k = 0; k < m; k++){    // Another loop gets executed m times
        print(k)
}
```

T(n,m) = **$\Theta(n^2 + m) = \Theta(n^2)$**

Data Structures and Algorithms

Geetha Lekshmy V
Aswathy R Nair
Department of Computer Science
Amrita Vishwa Vidyapeetham

Analysis of Recursive Algorithm – Factorial

```
fact(n){
    if (n == 0)  → base case
      return 1;
    else
      return n * fact(n - 1);  → recursive call
  }
```

For every recursion there should be a base case, which specifies the termination of recursion.

With out base condition recursion will not terminate.

Analysis of Recursive Algorithm –Factorial

```
fact(n) {
    if (n == 0)    return 1;
    else
       return n * fact(n - 1);
  }
```

Eg:- fact(4) →24

4 * fact(3)→6

3 * fact(2) →2

2*fact(1) → 1

1*fact(0) →1

Basic operation - operations in recursive statement

T(0) = 0 → initial condition

When n=0 there are no multiplication operation.

T(n) = T(n-1) + 1→ recurrence relation

Analysis of Recursive Algorithm – How?

1. Problem Size
2. Identify basic operation(operation in recursive call)
3. Write the recurrence relation, Initial conditions
4. Solve the recurrence relation

(Recurrence Relation -A recurrence relation is an equation that defines a sequence based on a rule that gives the next term as a function of the previous term(s).)

1Ans:- n
2 Ans:- multiplication
3 Ans:- T(n)=T(n-1)+1, T(0)=0

Analysis of Recursive Algorithm

4 Ans:-Solve the recurrence relation

$T(n) = T(n-1) + 1$

$T(n-1) = T((n-1)-1)+1 = T(n-2) + 1$

$T(n-2) = T((n-2)-1)+1 = T(n-3)+1$

$T(n) = T(n-1) + 1$

$= [T(n-2) + 1)] + 1$

$= T(n-2) + 2$

$= T(n-3) + 1 + 2$

$T(n) = T(n-3) + 3$

...

substitution method

$T(n) = T(n-i) + i$ //ith recursive call

$T(n) = T(n-n) + n$

$= T(0) + n$

$= 0 + n = n$

$T(n) = \theta(n)$

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Analysis of Recursive Algorithm - Maximum element

```
Max(A[ ], n) {
if (n == 1)
        return A[0];
 else{
      temp = Max(A, n-1 );
      If (temp < A[n − 1])
         return A[n − 1];
      else
        return temp;
     }
}
```

A= {23,10,60,45}
Max(A,4)

60

temp=Max(A,3)→

```
Max(A,3){
```

45

```
 if(temp < A[3])
    return A[3] }
else
```

60

```
   return temp
```

23

```
 temp=Max(A,2);
```

→

```
Max(A,2){
```

23 < 60

```
if (temp<A[2])
```

60

```
return A[2]

else return temp
```

23

```
temp=Max(A,1)

If( temp< A[1])
```

10

```
   return A[1]
else
```

23

```
    return temp
}
```

```
Max(A,1)
{
If(n==1)
```

23

```
 return A[0]
}
```




Analysis of Recursive Algorithm - Maximum element

```
Max(A[ ], n){
if (n == 1)
        return A[0];
 else{
      temp = Max(A, n-1 );
      If (temp < A[n − 1])
         return A[n − 1];
      else
        return temp;
    }
}
```

- Specify problem size - n
- Identify basic operation(operation in recursive call) - 1 assign+ 1 call+ 1 decr+1 comparison + 1 indexing+ 1 return
- Write the recurrence relation, Initial conditions

  Initial Condition – when n==1, 1 indexing + 1ret

  T(1) = 2

  Recurrence Relation

  T(n) = T(n-1) + 6

  So,

  T(1) = 2 if n=1

  T(n)= T(n-1)+6 otherwise

- Solve the recurrence relation

Analysis of Recursive Algorithm - Maximum element

```
Max(A[ ], n){
if (n == 1)
        return A[0];
 else{
      temp = Max(A, n-1
);
      If (temp < A[n − 1])
         return A[n − 1];
       else
         return temp;
     }
 }
```

- Solve the recurrence relation

T(n) = 2 if n=1

T(n)= T(n-1)+6 otherwise

T(n) = T(n-1)+6

= (T(n-2)+6) +6

= T(n-3)+6+ 6+ 6

T(n) =T(n-3)+3 * 6

In ith step

T(n) = T(n-i)+i*6 ….

T(n)= T(n-(n-1))+ (n-1)*6

=T(1)+6n- 6

= 2+6n -6 =6n-4

T(n) = θ(n)

This method of solving recurrence – **Substitution method**

T(n-1)=T((n-1)-1)+6

= T(n-2)+6

T(n-2)=T((n-2)-1)+6

= T(n-3)+6



Analysis of Recursive Algorithm –Binary Search

```
bSearch(A[],k,l,h){
  if(l > h){
        return -1;
   }
   mid=(l+ h)/2
   if(a[mid]==k)
      return mid
   else if(a[mid]>key)
      return bSearch(A,k,l,mid-1)
  else
      return  bSearch(A,k,mid+1,h)}
}
```

$T(n) = T(n/2) + c , n > 1$

$T(1) = 2$

$T(n) = \theta(\log n)$ (Solved by Master's method)

{ $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$, constant, so $\theta(\log n)$ **}**

Summary

Algorithm analysis is estimating the run time of an algorithm, by counting the number of basic operations in it and expressing it in terms of input size.

Linear search has its running time as a linear function of input size.

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

